

基于 CUDA 纹理插值的空间多目标交会序列快速规划方法

刘 浩*, 贺子鹏, 徐广德

(北京空间飞行器总体设计部, 北京 100094)

摘要: 针对卫星对多个异面目标连续交会的序列规划问题, 提出基于计算统一设备架构 (compute unified device architecture, CUDA) 并行计算的快速规划方法。该方法以目标穿越卫星轨道面时刻为交会时机, 首先形成交会候选方案集合, 将目标间转移速度增量计算简化为基于转移速度和飞行角的双线性插值问题; 然后利用纹理内存硬件插值快速估算速度增量, 通过多线程块实现候选方案层面的并行化, 并在单个候选方案内部通过并行规约和共享内存实现目标间转移速度增量计算的并行化以及总速度增量累加; 最终对方案遍历寻优, 实现速度增量最小化的多目标交会序列规划。仿真结果表明, 此方法能够快速规划最优交会序列, 相比串行计算可加速 20 倍以上, 在智能算力逐步向太空普及的趋势下, 未来可大幅提升多目标观测等场景的在轨规划效率。

关键词: 任务规划; 多目标序列规划; 并行计算; 计算统一设备架构

中图分类号: V 52

文献标志码: A

DOI: 10.12305/j.issn.1001-506X.2026.04.21

Fast planning method for space multi-target rendezvous sequence based on CUDA texture interpolation

LIU Hao*, HE Zipeng, XU Guangde

(Beijing Institute of Spacecraft System Engineering, Beijing 100094, China)

Abstract: Aiming at the satellite sequence planning problem of continuous rendezvous with multiple noncoplanar targets, a fast planning method based on compute unified device architecture (CUDA) is proposed. The rendezvous timing is taken as the moment when the target crosses the orbital plane of the satellite. First, a set of candidate solutions for the rendezvous sequence is formed, and the calculation of the transfer velocity increment between targets based on the orbital dynamics model is simplified to a bilinear interpolation problem based on the transfer velocity and the flight angle. Then, texture memory is applied for fast estimation of the velocity increment, the parallelization at the level of candidate plan is realized by multiple thread blocks, and the parallelization of the transfer velocity increment calculation between targets and the accumulation of the total velocity increment are realized by parallel reduction and shared memory within a single candidate plan. Finally, an exhaustive search optimization approach is applied to the candidate solutions and the multi-target rendezvous sequence planning for minimizing the velocity increment is achieved. Simulation results show that this method can quickly plan the optimal rendezvous sequence, which is more than 20 times faster than serial calculation. As intelligent computing power gradually becomes more widespread in space, it is expected to significantly improve the efficiency of on-orbit planning for scenarios such as multi-target observation in the future.

Keywords: mission planning; multiple-target sequence planning; parallel computing; compute unified device architecture (CUDA)

收稿日期: 2024-12-04; 修回日期: 2025-02-06; 网络优先出版日期: 2025-05-20。

网络优先出版地址: <https://link.cnki.net/urlid/11.2422.tn.20250520.1821.022>

* 通讯作者。

引用格式: 刘浩, 贺子鹏, 徐广德. 基于 CUDA 纹理插值的空间多目标交会序列快速规划方法[J]. 系统工程与电子技术, 2026, 48(4): 1331-1339.

Reference format: LIU H, HE Z P, XU G D. Fast planning method for space multi-target rendezvous sequence based on CUDA texture interpolation[J]. Systems Engineering and Electronics, 2026, 48(4): 1331-1339.

0 引言

多目标交会序列规划是在轨加注服务、空间碎片清除、多目标连续观测等任务场景需解决的共性问题^[1-4], 卫星在指定的时间范围内以一定顺序在特定的多个时刻连续与多个目标进行交会, 规划的目的通常是寻找到一个特定序列作为最优方案, 使卫星变轨策略在满足各类约束条件的情况下, 消耗的速度增量或其他代价最小^[5]。

多目标序列规划问题的难点是当目标数量较多、交会时刻选择自由度较大时, 序列方案数量将十分巨大, 且每个序列内部的目标间转移过程需利用复杂的轨道动力学方法分步计算速度增量, 通过遍历所有可能的交会序列方案以寻找最优序列。针对上述问题, 需研究相对简化的快速序列规划方法, 以实现运行时间可接受的在轨或地面规划。

对多目标交会序列规划问题的研究主要集中在两方面, 一是目标间转移速度增量求解或估算方法, 二是在此基础上的多目标交会序列规划方法。

目标间转移速度增量计算问题本质上是微分方程两点边界值的 Lambert 问题, 即给定初末位置和转移时间, 求初末速度^[6]。Lambert 问题应用广泛, 一直是航天器轨道交会领域的热点问题^[7], 只有特定类型的 Lambert 问题可以找到解析解, 在实际应用中数值求解通常是更灵活的方法^[8]。Davis 等^[9]对能量最优的 Lambert 问题进行研究, 提出解析闭式解; 徐利民等^[10]针对能量最优与燃料最优 Lambert 双脉冲交会问题提出一种基于矢量形式的解析计算方法; 黄勇等^[11]针对初始位置、目标位置和转移时间均不固定的双冲量 Lambert 问题, 提出一种遗传算法与迭代相结合的数值求解方法。综合来看, 解析法通常涉及到复杂的数学运算和近似处理, 需要进行多项式方程求解等, 且适应范围有限; 数值解法则需要迭代计算等耗时较多的步骤, 难以应用于求解次数多、效率要求高的场景。

考虑到目标数量较多情况导致的规划计算次数大幅增加问题, 目前的多目标交会序列规划一般采用近似计算方法和空间搜索方法^[12-13]降低时间开销。Gatto 等^[14]基于平面内和平面外推力角的最优控制律近似, 扩展了经典的 Edelbaum 近似估计法使其能够处理倾角变化较小的转移, 提高了计算速度, 将其应用于多天体访问场景。Zhu 等^[15]提出一种基于多层感知器学习的快速评估小推力转移的方法, 设计了一种高效的数据库生成方法用于获取不可行转移和最优转移。Bando 等^[16]研究了转移时间间隔固定的多近地小行星交会任务规划问题, 提出基于生成函数的交会序列优化方法, 但该方法仅适用于目标较少的场景。Kong 等^[17]针对多个空间机器人访问多个异面航天器的优化问题, 瞄准目标飞过其轨道升降交点时进行巡视, 提出的优化方法使用了两个嵌套循环, 外环求解器用于优化机器人位置、任务分配和访问顺序, 内环求解器用于找到最佳轨迹, 采用遗传算法解决外环问题, 采用一种分支和约束算法

快速解决内环问题。Zhou 等^[18]研究了带有服务卫星和燃料站的多颗地球静止轨道 (geostationary Earth orbit, GEO) 卫星加注任务的调度问题, 将加注顺序和二元决策变量作为优化变量, 总燃料消耗作为设计目标, 建立了双层优化模型, 上层遗传算法优化加注顺序, 下层随机搜索快速定位近似最优二元决策变量。

近年来, 基于英伟达图形处理器 (graphics processing unit, GPU) 和计算统一设备架构 (compute unified device architecture, CUDA) 开发套件的并行计算技术已经大规模应用于以深度学习为代表的人工智能^[19-20]、以光线追踪^[21-22]和深度学习超采样^[23-24]为代表的游戏开发等领域, 对于数值仿真^[25]、任务规划^[26-29]等科学计算问题, CUDA 也逐步开始应用, 大大提高了计算效率。本文针对异面多目标交会问题, 以不改变自身轨道面、瞄准目标穿越卫星轨道面时刻为节省速度增量的基本前提, 深入结合 CUDA 内存架构特性以及并行计算原理, 提出以飞行速度和飞行角为特征参数的目标间转移速度增量插值估算方法, 充分利用多项 CUDA 并行计算特性, 对巨量序列方案及其内部多段转移过程进行并行计算, 以支持对序列方案的遍历寻优, 实现了计算效率高、精度可接受的多目标交会序列规划。

1 异面多目标交会问题

本文异面多目标交会问题针对的是地球同步轨道上的卫星及 N 个目标, 具有不同的倾角和定点经度, 记卫星和多个目标的定点经度分别为 Ω 和 $\{\Omega_n, 1 \leq n \leq N\}$, 其中 n 为目标编号。

如图 1 所示, 每个目标均按照双周期性 (每两次为一个周期) 在轨道周期内的两个特定相位穿越卫星轨道面, 卫星在指定的时间范围内对每个目标均选择一个穿越时刻, 连续转移至对应相位, 即实现了对异面多目标的连续交会, 图 1 中卫星按照目标编号顺序进行交会。

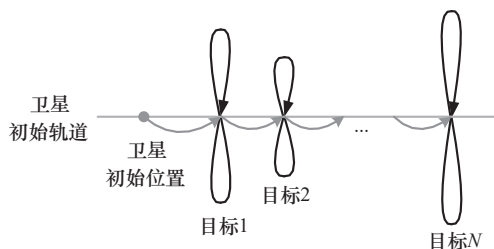


图 1 异面多目标交会问题示意图

Fig. 1 Schematic diagram of rendezvous problem between multiple noncoplanar targets

1.1 穿越时刻计算及交会方案分析

每个目标在指定时间范围内穿越卫星轨道面时刻的计算方法如图 2 所示, 借助卫星和每个目标轨道面的交线确定首次交会时卫星的真近点角, 进而确定过近地点时间和绝对时间, 最后基于双周期性递推计算所有穿越时刻。

基于上述方法, 可计算得到指定时间范围内所有目标对卫星轨道面的穿越时刻集合, 记为 $\{t_{n, \text{crossing}} = \{t_1^n, t_2^n, \dots, t_m^n, \dots, t_M^n\} | 1 \leq n \leq N\}$, 假设指定的时间范围为整数天 N_{days} , 则

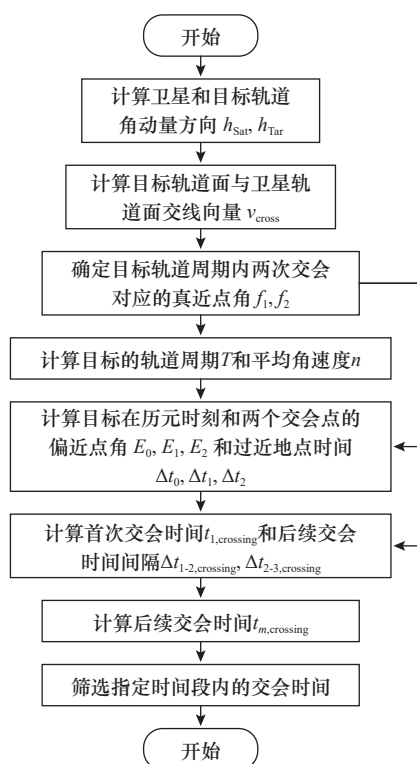


图2 目标穿越卫星轨道面时刻计算方法

Fig. 2 Method for calculating the time when the target crosses the orbital plane of satellite

每个目标的穿越时刻数量均为 $M = 2N_{\text{days}}$ 。

以时间为横坐标, 以卫星和目标的定点经度为纵坐标, 时间范围限定至 4.5 天, 所有目标对卫星轨道面穿越时刻集合的计算结果如图 3 所示, 图中不同类型的散点表示不同目标, 初始时刻和经度的卫星位于左下方。

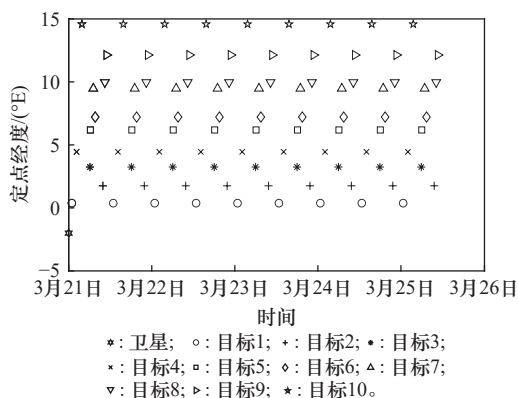


图3 多目标穿越卫星轨道面时刻示意图

Fig. 3 Schematic diagram of the timing of multiple targets crossing the satellite orbital plane

在图 3 中, 对每个目标各选一个穿越时刻, 连同卫星初始时刻一起按照时间先后顺序连接, 即为一种交会序列方案。由此可知, 对于上述异面多目标交会问题, 交会序列方

案总数量为 M^N , 以上标 i 表示方案索引, 定义所有交会序列方案的集合为

$$\{\text{seq}^i = \{p_1^i, p_2^i, \dots, p_j^i, \dots, p_N^i\} | 1 \leq i \leq M^N\} \quad (1)$$

式中: $p_j^i = \{s_j^i, t_j^i, \Omega_j^i, \Delta v_j^i\}$ 为第 i 个交会序列中第 j 次交会的参数集合, 具体包括目标编号 s_j^i 、交会时刻 t_j^i 、目标相位 Ω_j^i 以及速度增量 Δv_j^i 。

1.2 目标间转移速度增量递推计算方法

对于交会序列 seq^i 中的第 j 次转移, 也即卫星从目标 s_{j-1}^i 转移至目标 s_j^i 与其交会、然后转移至目标 s_{j+1}^i 的过程, 转移前后的速度大小、方向与速度增量的关系存在两种典型模式: ①轨道同侧转移: 转移前后均位于卫星原始轨道高度以下或以上; ②轨道异侧转移: 转移时穿越卫星原始轨道。两种模式的目标间转移示意图分别如图 4(a) 和图 4(b) 所示。

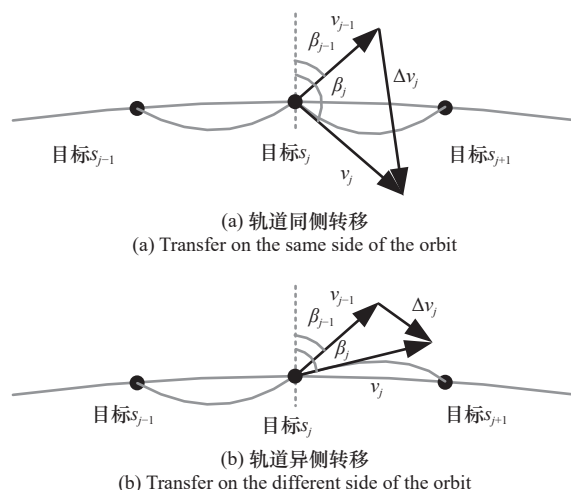


图4 交会序列中目标间转移示意图

Fig. 4 Schematic diagram of transfer between targets in a rendezvous sequence

已知第 j 次转移对应的交会时刻 t_j^i 和目标相位 Ω_j^i , 以及第 $j+1$ 次转移对应的交会时刻 t_{j+1}^i 和目标相位 Ω_{j+1}^i , 通过 Lambert 问题求解方法可计算转移所需的速度增量 Δv_j^i 。然而, Lambert 方法计算过程较为复杂, 且第 j 次转移计算需以第 $j-1$ 次转移的计算结果为输入, 交会序列内部每个转移过程的求解必须完全串行实现, 考虑到交会序列方案数量巨大, 采用 Lambert 求解方法并完全串行计算, 其运行时间对于工程应用是不可接受的。

本文提出利用飞行速度和飞行角作为特征参数表征转移状态, 以实现递推计算速度增量的方法。如图 4 所示, 飞行角 β 定义为速度方向与轨道径向的夹角。假设上次转移已获得大小为 v_{j-1}^i 、方向为 β_{j-1}^i 的速度, 本次转移后需具备大小为 v_j^i 、方向为 β_j^i 的速度, 则本次转移所需的速度增量大小 Δv_j^i 可利用如下三角形余弦定理计算:

$$\Delta v_j^i = \sqrt{v_{j-1}^2 + v_j^2 - 2v_{j-1}v_j \cos(|\beta_{j-1} - \beta_j|)} \quad (2)$$

对于上式,可一般化为函数关系式 $\Delta v_j^i = f_{\Delta v}(v_{j-1}, \beta_{j-1}, v_j, \beta_j)$,因此可通过前后两次转移的飞行速度和飞行角特征参数描述目标间转移过程,将对 N 个目标的连续交会过程分割为 N 个独立的转移过程,即“初始位置 $\rightarrow$ 目标 s_1^i ”...“目标 $s_j^i \rightarrow$ 目标 s_{j+1}^i ”...“目标 $s_{N-1}^i \rightarrow$ 目标 s_N^i ”,对交会序列中的 N 个转移过程实现了解耦。

对于完整的交会序列,考虑转移前后飞行角的对称性,可通过下式递推计算每个转移过程的速度增量:

$$\begin{cases} \Delta v_j = \sqrt{v_{j-1}^2 + v_j^2 - 2v_{j-1}v_j \cos(|\beta_{j-1} - \beta_j|)} \\ v_0 = \sqrt{\frac{GM}{R}}, \beta_0 = \frac{\pi}{2} \end{cases} \quad (3)$$

式中: v_0 和 β_0 为卫星初始情况下的速度和飞行角; $G=6.674 \times 10^{-11} \text{ N} \cdot (\text{m/kg})^2$ 为引力常数; M 为地球质量; R 为卫星轨道半径。

1.3 飞行速度和飞行角插值估算方法

对于已知的轨道高度,卫星在两个目标间的转移过程可等效为以一定的时间间隔进行一定的相位调整。对于交会序列 seq_i 中卫星从目标 s_j^i 至目标 s_{j+1}^i 的单步转移过程,对应的时间差为 $\Delta t_{j \rightarrow j+1}^i = t_{j+1}^i - t_j^i$,相位差为 $\Delta \Omega_{j \rightarrow j+1}^i = \Omega_{j+1}^i - \Omega_j^i$,即为图3中某个交会序列对应的折线上前后两个散点间的横坐标差和纵坐标差。

本文在深度挖掘 GPU 特别是 CUDA 架构并行计算特性的基础上,提出以二维插值方法估算目标间转移速度增量的方法。定义二维插值函数 $g_{\text{interp2D}}()$,其自变量为转移时间 $\Delta t_{j \rightarrow j+1}^i$ 和转移相位 $\Delta \Omega_{j \rightarrow j+1}^i$,因变量为转移速度 v_j^i 和飞行角 β_j^i 两个特征参数,如下式:

$$\begin{cases} v_j^i = g_{\text{interp2D}}(\Delta t_{j \rightarrow j+1}^i, \Delta \Omega_{j \rightarrow j+1}^i | v_{N_{\Delta t} \times N_{\Delta \Omega}}^{\text{Table2D}}) \\ \beta_j^i = g_{\text{interp2D}}(\Delta t_{j \rightarrow j+1}^i, \Delta \Omega_{j \rightarrow j+1}^i | \beta_{N_{\Delta t} \times N_{\Delta \Omega}}^{\text{Table2D}}) \end{cases} \quad (4)$$

式中: $v_{N_{\Delta t} \times N_{\Delta \Omega}}^{\text{Table2D}}$ 和 $\beta_{N_{\Delta t} \times N_{\Delta \Omega}}^{\text{Table2D}}$ 分别为飞行速度和飞行角对应的插值表,可通过 Lambert 求解方法提前精确计算,同时计算次数相比序列方案数量和目标数量的乘积 $M^N \times N$ 相比大大减少; $N_{\Delta t}$ 和 $N_{\Delta \Omega}$ 分别为插值表在转移时间和转移相位两个维度的索引数量,插值索引分别记为 $\{\Delta t_1, \Delta t_2, \dots, \Delta t_{N_{\Delta t}}\}$ 和 $\{\Delta \Omega_1, \Delta \Omega_2, \dots, \Delta \Omega_{N_{\Delta \Omega}}\}$ 。

2 CUDA 并行计算方法

在目标间转移过程解耦以及速度增量二维插值求解方法的基础上,进一步结合 CUDA 内存架构以及并行计算逻辑,设计深度契合 CUDA 并行计算逻辑并充分利用硬件加速特性和内存特性的序列规划方法。图5为 CUDA 线程的多层级组织结构以及内存架构^[30]的示意图,其中包含中央处理器(central processing unit, CPU)。

在 CUDA 架构中,最小的并行计算单元是线程,每个线程可以独立执行指令;线程块是一组线程的集合,这些线程在同一处理器核心上执行,可以进行同步操作,并且能够访

问同一个共享内存空间;线程网格由多个线程块组成,可以跨多个处理器核心分布式执行,网格中的线程块可以并行执行,但不同线程块之间的线程仅能通过全局内存进行数据交换。

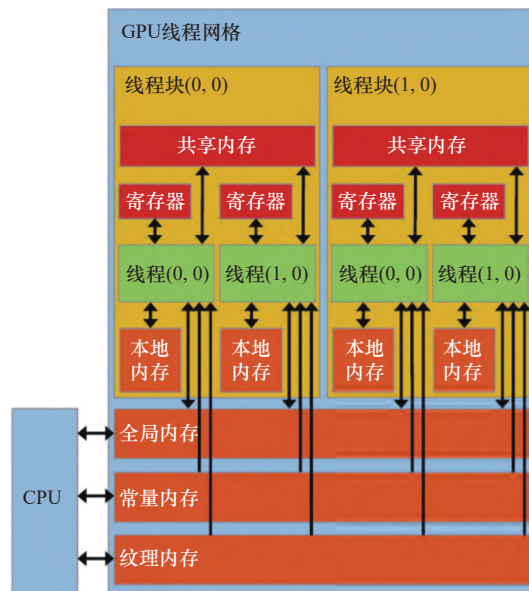


图5 CUDA 线程组织结构及内存架构示意图

Fig. 5 Schematic diagram of thread structure and memory architecture of CUDA

2.1 并行逻辑设计

为了最大限度提高并行化程度,提高整体计算效率,为所有交会序列方案中的所有目标间转移过程配置单独的线程,具体并行计算方案如下:

(1)不同交会序列方案之间不存在直接关联,无需高维的组织形式,因此采用一维线程网格,其 x 方向维度为 M^N ,即共配置 M^N 个线程块用于计算不同的交会序列方案;

(2)在每个交会序列方案内部,借助飞行速度和飞行角两个特征参数作为中间变量,经过 N 次递推计算速度增量并进行累加,最终得到交会序列方案所需的总速度增量,因此需为每个线程块配置至少 N 个线程,考虑到 CUDA 线程调度机制、内存访问效率以及后续并行规约计算的便利性, CUDA 线程块的维度通常是 2 的幂次方,因此采用一维线程块,其 x 方向维度为线程数量 Q ,并满足下式:

$$Q = 2^q \geq N \quad (5)$$

式中: q 为正整数,例如目标数量 $N=10$ 时线程数量 Q 取 16。

基于以上并行计算方案,设置 CUDA 核函数及其启动参数如下:

$$\text{Kernel} \langle \langle M^N, Q \rangle \rangle$$

2.2 基于纹理内存硬件插值实现速度增量计算

CUDA 纹理内存最有用的特性是能够实现基于硬件的数据自动插值,相比软件插值速度更快。纹理内存支持最近邻插值和线性插值两种算法,特别是对于二维和三维数组分别支持硬件级别的双线性和三线插值^[31-32]。此外,纹

理内存还能够自动处理插值时自变量索引超出边界的情况,不需要任何特殊的处理代码就能做到数组访问永不越界,可设置超出边界时的默认值。

结合式(4)所述计算原理,使用 CUDA 纹理内存硬件插值实现目标间转移速度增量计算的算法流程如图 6 所示。

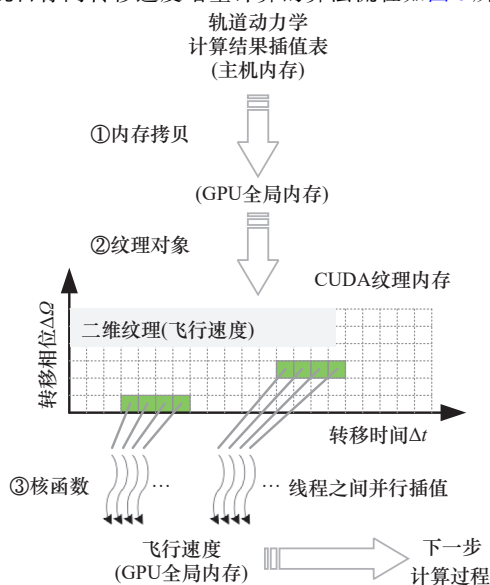


图 6 基于纹理内存硬件插值计算速度增量的计算流程

Fig. 6 Calculation process of velocity increment based on hardware interpolation in texture memory

首先创建纹理对象,通过纹理描述符设定插值方式为双线性插值,纹理寻址模式为指定边界值,使用非归一化纹理坐标,在核函数中通过下式分别计算转移时间和转移相位维度的插值索引:

$$\begin{cases} \Delta t_{\text{Kernel}} = \frac{\Delta t - \Delta t_1}{\Delta t_{N_{\Delta t}} - \Delta t_1} \cdot (N_{\Delta t} - 1) \\ \Delta \Omega_{\text{Kernel}} = \frac{\Delta \Omega - \Delta \Omega_1}{\Delta \Omega_{N_{\Delta \Omega}} - \Delta \Omega_1} \cdot (N_{\Delta \Omega} - 1) \end{cases} \quad (6)$$

式中: Δt 和 $\Delta \Omega$ 为实际尺度的转移时间和转移相位; $\Delta t_{\text{Kernel}} \in [0, N_{\Delta t} - 1]$ 和 $\Delta \Omega_{\text{Kernel}} \in [0, N_{\Delta \Omega} - 1]$ 为硬件插值采用的非归一化自变量索引。

2.3 基于线程块内并行规约实现速度增量累加

除了对不同线程块即不同交会序列方案实现了并行化计算,本文进一步设计了线程块内部线程之间的并行规约算法,将理论并行线程数量从 M^N 提升至 $M^N \times N$ 。

一般地,目标数量为 N 时,在 CPU 中运行的传统串行累加算法需先后计算 N 次加法,具体计算流程如算法 1 所示。

算法1 CPU串行累加算法

```

 $\Delta v_{\text{SUM}} \leftarrow 0$ 
for  $j$  from 0 to  $N-1$  do
 $v_j = \text{ginterp2D}(\Delta t_{j \rightarrow j+1}, \Delta \Omega_{j \rightarrow j+1} | v_{N_{\Delta t} \times N_{\Delta \Omega}}^{\text{Table2D}})$ 
 $\beta_j = \text{ginterp2D}(\Delta t_{j \rightarrow j+1}, \Delta \Omega_{j \rightarrow j+1} | \beta_{N_{\Delta t} \times N_{\Delta \Omega}}^{\text{Table2D}})$ 
 $\Delta v_j = f_{\Delta v}(v_{j-1}, \beta_{j-1}, v_j, \beta_j)$ 
 $\Delta v_{\text{SUM}} \leftarrow \Delta v_{\text{SUM}} + \Delta v_j$ 

```

并行规约算法可将计算次数从 N 次减少至 q 次(按照式(5), $Q = 2^q \geq N$),并在 GPU 中完成交会序列内部各个转移过程的速度增量累加。线程块中第 j 个线程中所运行的核函数算法如算法 2 所示。

算法2 CUDA并行规约累加算法

```

 $v_j = \text{gtex2D}(\Delta t_{j \rightarrow j+1}, \Delta \Omega_{j \rightarrow j+1} | v_{N_{\Delta t} \times N_{\Delta \Omega}}^{\text{Table2D}})$ 
 $\beta_j = \text{gtex2D}(\Delta t_{j \rightarrow j+1}, \Delta \Omega_{j \rightarrow j+1} | \beta_{N_{\Delta t} \times N_{\Delta \Omega}}^{\text{Table2D}})$ 
 $v_{\text{prev}}^{\text{Shared}}[j+1] = v_j$ 
 $\beta_{\text{prev}}^{\text{Shared}}[j+1] = \pi - \beta_j$ 
__syncthreads()
 $v_{j-1} = v_{\text{prev}}^{\text{Shared}}[j]$ 
 $\beta_{j-1} = \beta_{\text{prev}}^{\text{Shared}}[j]$ 
 $\Delta v_j = f_{\Delta v}(v_{j-1}, \beta_{j-1}, v_j, \beta_j)$ 
 $\Delta v_{\text{Shared}}[j] = \Delta v_j$ 
__syncthreads()
 $k = Q/2$ 
for  $j$  from 0 to  $p-1$  do
 $\Delta v_{\text{Shared}}[j] \leftarrow \Delta v_{\text{Shared}}[j] + \Delta v_{\text{Shared}}[j+k]$ 
 $k \leftarrow k/2$ 
__syncthreads()

```

其中, $v_{\text{prev}}^{\text{Shared}}$ 、 $\beta_{\text{prev}}^{\text{Shared}}$ 以及 Δv_{Shared} 均为保存在线程块中的共享内存变量,均是长度为 Q 的数组,前两者的目的是通过共享内存实现前后两个转移过程飞行速度和飞行角特征参数的传递,后者则是为了支持并行规约累加;同步函数 __syncthreads() 的目的是等待线程块中所有线程均已完成前面的计算。

所有线程执行完成后,对应序列的总速度增量累加结果 Δv_{SUM} 保存在共享内存数组 Δv_{Shared} 的第一个变量中,即 $\Delta v_{\text{Shared}}[0]$ 。基于线程块内并行规约实现速度增量累加的示意图如图 7 所示。

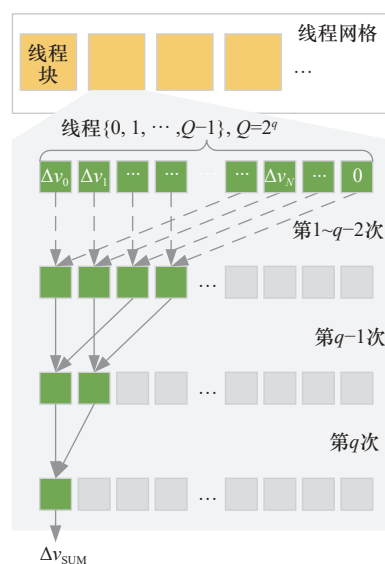


图 7 线程块内并行规约实现速度增量累加示意图

Fig. 7 Parallel reduction within thread blocks to achieve velocity increment and accumulation

2.4 序列规划算法运行流程

空间多目标交会序列快速规划算法的流程如图 8 所示, 采用典型的混合编程结构, 主体部分(虚线框内)在 GPU 内进行计算, CPU 完成数据准备和双向内存拷贝(从 CPU 拷贝输入数据至 GPU, 从 GPU 拷贝计算结果至 CPU)。

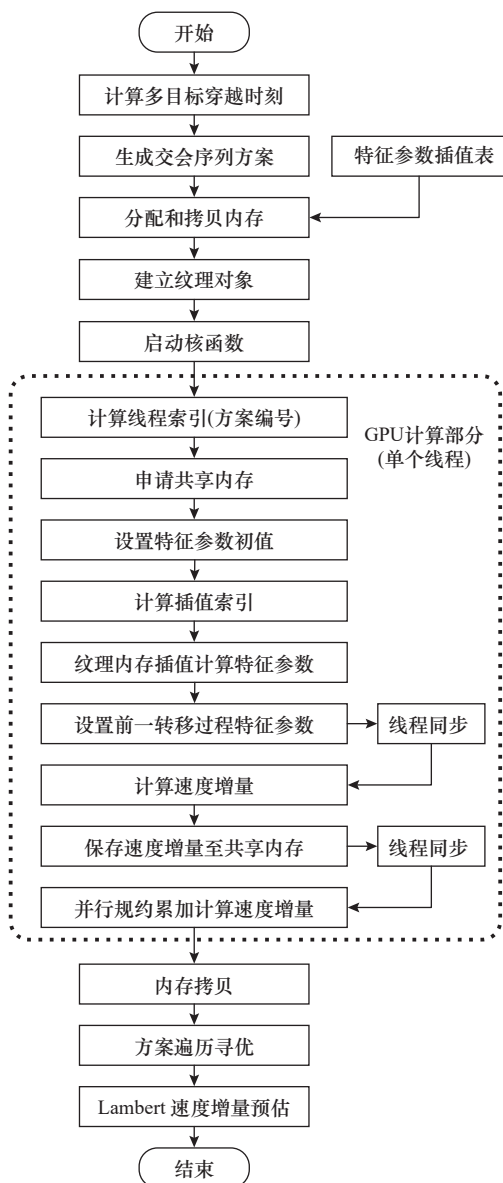


图 8 空间多目标交会序列快速规划算法流程

Fig. 8 Process of fast planning algorithm for space multi-target rendezvous sequence

图中虚线框内为 CUDA 单个线程的运算内容, 与 CPU 串行算法相比, 特殊的是须通过多次线程同步实现线程块内所有线程中关键计算步骤的同步, 以支持通过共享内存读写实现线程间数据联动, 防止未完成线程未初始化数据引起的计算错误。

将所有方案的速度增量从 GPU 取回后, 通过遍历比较寻优确定最优序列, 并使用 Lambert 算法对最优序列进行规划, 得到速度增量预估值。

3 仿真结果与分析

设定多目标交会的时间范围要求为 4 天、4.5 天以及 5 天, 共 3 个仿真场景。卫星和 10 个目标的轨道倾角均在 15° 以内, 目标定点经度在 $0^\circ\text{E}\sim 15^\circ\text{E}$ 范围内, 卫星初始经度为 -1.96°W , 与距离最近的目标相距约 2° , 目标的倾角、定点经度、真近点角取随机值。仿真场景中卫星和多目标的具体参数设置如表 1 所示。

表 1 仿真场景卫星和目标轨道参数设置

Table 1 Orbital parameter settings for satellites and targets in simulation scenarios

名称	倾角/ $^\circ$	定点经度/ $^\circ\text{E}$	真近点角/ $^\circ$
卫星	5.0	-1.96	0
目标1	7.1	0.22	155.31
目标2	2.8	1.75	327.83
目标3	13.2	3.83	65.46
目标4	4.2	4.44	94.97
目标5	8.9	6.53	52.39
目标6	14.6	8.21	48.98
目标7	5.1	9.38	312.95
目标8	9.3	10.30	208.69
目标9	2.7	12.17	197.95
目标10	5.3	14.70	52.18

根据时间范围和目标经度范围, 生成飞行速度和飞行角两个插值表, 转移时间和转移相位均为等间隔, 索引数量分别为 8 和 21。对于不同仿真场景, 使用本文所提出的多目标交会序列快速规划方法分别计算最优序列及其对应的速度增量, 并基于最优序列的交会次序和交会时间使用 Lambert 算法再次计算速度增量, 以后者为基准计算百分比误差, 如表 2 所示, 3 个场景的速度增量计算结果较为接近, 误差均在 3% 以内, 验证了本文算法对速度增量估算的准确性。

表 2 不同仿真场景下的速度增量计算误差对比

Table 2 Comparison of velocity increment calculation errors in different simulation scenarios

	4天	4.5天	5天
交会序列			
本文最优序列	648.56	487.11	369.08
Lambert最优序列	666.24	493.04	374.82
Lambert顺序序列	1 313.54	723.26	520.44

进一步,针对3个场景分别设定直接按目标编号顺序进行交会的序列作为非寻优方案,通过Lambert算法计算速度增量,与最优序列速度增量相比,如表1所示,节省比例分别为49.28%、31.83%以及27.98%。可以看出,最优序列可节省相当体量的速度增量,且时间范围要求越短,通过序列规划以获得最优序列的必要性越显著,4天场景下最优序列

比直接顺序交会相比可节省约一半速度增量。

对于4.5天场景,最优交会序列中对每个目标的交会顺序、交会时间以及转移所需速度增量如表3所示,将本文方法计算的速度增量与Lambert算法计算结果对比,可以看出每一段目标间转移的速度增量预估结果误差较小,基本保持在5%以内。

表3 最优交会序列的多目标交会时间以及速度增量计算结果对比

Table 3 Comparison of multi-target rendezvous time and velocity increment calculation results for optimal rendezvous sequence

序号	交会目标	交会时间	本文速度增量/(m/s)	Lambert速度增量/(m/s)	误差/%
1	目标5	2027年3月21日 18:14:25.080	41.23	42.98	4.07
2	目标4	2027年3月22日 02:12:13.283	18.35	18.52	0.92
3	目标7	2027年3月22日 19:00:50.072	28.24	27.90	1.22
4	目标8	2027年3月22日 22:20:54.968	93.05	92.27	0.85
5	目标6	2027年3月23日 07:31:46.070	55.67	55.58	0.16
6	目标2	2027年3月23日 21:43:02.576	99.87	101.94	2.03
7	目标3	2027年3月24日 05:57:01.641	9.72	9.88	1.62
8	目标1	2027年3月24日 12:41:03.313	84.11	86.51	2.77
9	目标10	2027年3月25日 03:33:10.517	20.57	21.85	5.86
10	目标9	2027年3月25日 10:40:54.521	36.30	35.61	1.94
交会序列总速度增量/(m/s)			487.11	493.04	1.20

下面分析本文方法对计算效率的提升效果。基于CUDA并行计算的空间多目标交会序列快速规划算法全流程以及主要步骤的运行时间如表4所示,用于对比计算效率的CPU串行算法经过优化,在遍历序列方案的过程中采取了剪枝手段以避免对所有方案及其内部的目标间转移过程完全遍历。分析表中结果,3个场景下使用本文方法获得的加速比均在20倍以上。序列方案数量越多的场景,CUDA计算的并行度越高,加速比越大。

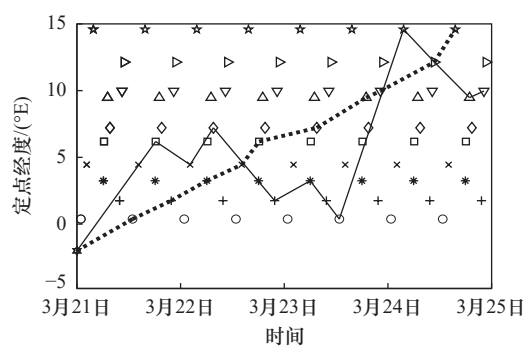
表4 不同仿真场景下的算法运行时间对比

Table 4 Comparison of algorithm running time in different simulation scenarios

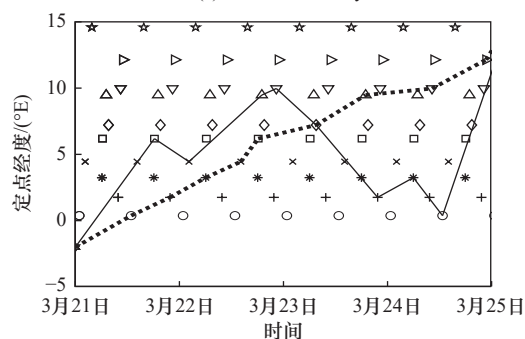
算法名称	步骤	4天	4.5天	5天
CPU串行算法	-	221.50	867.95	2910.79
本文算法	步骤1(CPU至GPU内存拷贝)	0.37	0.54	0.59
	步骤2(使核函数并行计算)	8.77	31.59	105.46
	步骤3(GPU至CPU内存拷贝)	0.16	0.19	0.22
	全流程	9.3	32.32	144.27
加速比/%		23.82	26.85	27.39

最后,分析序列规划方法得出的最优序列与直接顺序交会相比节省速度增量的主要原理。不同任务时间要求的3个场景对应的最优交会序列和顺序交会序列分别如图9(a)~图9(c)所示,图中实线为最优交会序列,虚线为顺

序交会序列。



(a) 4天场景
(a) Scenario of 4 days



(b) 4.5天场景
(b) Scenario of 4.5 days

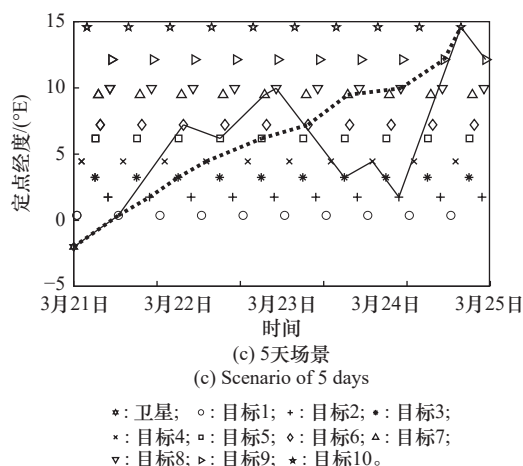


图 9 不同场景对应的最优和顺序交会序列示意图

Fig. 9 Schematic diagram of the optimal and ordinal rendezvous sequence in different simulation scenarios

以 4.5 天场景为例, 速度增量消耗最少的最优交会序列对应的目标间转移示意图如图 10 所示, 图中青色曲线为目标在 GEO 参考坐标系下的相对轨道, 红色曲线为卫星在 GEO 参考坐标系下的相对转移轨迹。

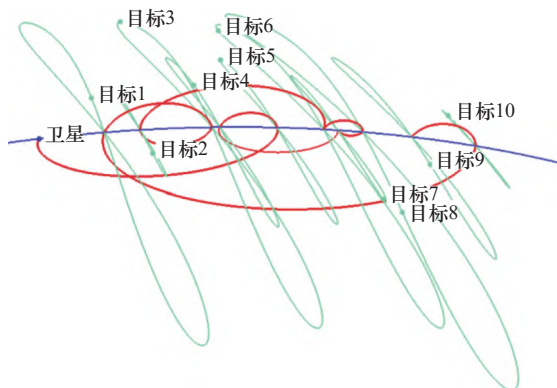


图 10 最优交会序列对应的卫星相对转移轨迹

Fig. 10 Relative transfer trajectory of the satellite corresponding to the optimal rendezvous sequence

分析图 9(b)和图 10 可知, 从完整交会序列来看, 最优序列充分利用目标的特定穿越时机, 采取恰当的迂回策略以实现多目标连续交会; 在交会序列内部, 对于具体两目标间的转移策略, 最优序列中的卫星在大多数情况下采取了穿越 GEO 轨道的策略, 即图 4(b)的轨道异侧转移情形, 与图 4(a)的轨道同侧转移情形相比, 变轨前后的速度方向位于同侧时, 速度增量的模值较小。

4 结论

对于空间异面多目标连续交会的序列规划问题, 本文提出了基于 CUDA 并行计算的快速规划方法, 在对目标间转移过程建立简化模型的基础上, 基于 CUDA 实现了并行计算, 并利用硬件插值、共享内存以及并行规约特性提高了

并行化程度, 与串行方法相比大幅提高了计算效率。文中给出的仿真实例说明了该方法的有效性。

参考文献

- [1] XU Y J, LIU X L, HE R J, et al. Active debris removal mission planning method based on machine learning[J]. *Mathematics*, 2023, 11(6): 1419.
- [2] DANESHJOU K, MOHAMMADI -DEHABADI A A, BAKHTIARI M. Mission planning for on-orbit servicing through multiple servicing satellites: a new approach[J]. *Advances in Space Research.*, 2017, 60(6): 1148–1162.
- [3] ANDREY A B, DMITRIY A G. Review of path planning in prospective multi-target active debris removal missions in low earth orbits[J]. *Progress in Aerospace Sciences*, 2024, 145: 100982.
- [4] SARAH J M, CIARA N M, OLIVIER L D. Optimization of multispacecraft maneuvers for mobile target tracking from low earth orbit[J]. *Journal of Spacecraft and Rockets*, 2023, 60(2): 581–590.
- [5] HAO Y T, LIU Z Y, XU S H. Minimal delta-v-based orbital rendezvous for approaching a non-cooperative target on GEO[C]// Proc. of the Chinese Conference on Swarm Intelligence and Cooperative Control, 2023: 567–579.
- [6] BAI S Z, WANG Y, LIU H Y, et al. Finite-thrust Lambert transfer based on multistage constant-vector thrust control[J]. *IEEE Trans. on Aerospace and Electronic Systems*, 2023, 59(5): 4947–4967.
- [7] AVANZINI G, PALMAS A, VELLUTINI E. Solution of low-thrust Lambert problem with perturbative expansions of equinoctial elements[J]. *Journal of Guidance, Control, and Dynamics*, 2015, 38(9): 1585–1601.
- [8] AVANZINI G. A simple Lambert algorithm[J]. *Journal of Guidance, Control, and Dynamics.*, 2008, 31(6): 1587–1594.
- [9] DAVIS J, AVENDANO M, MORTARI D. A closed-form solution to the minimum Δv_{TOT} Lambert's problem[J]. *Advances in the Astronautical Sciences*, 2010, 135: 1037–1052.
- [10] 徐利民, 张涛, 陶佳伟. 能量最优与燃料最优 Lambert 交会问题[J]. 北京航空航天大学学报, 2018, 44(9): 1888–1893.
- [11] 黄勇, 李小将, 张东来, 等. 混合遗传算法在最优 Lambert 轨道转移设计中的应用[J]. 飞行力学, 2013, 31(3): 269–272.
- [12] HUANG Y, LI X J, ZHANG D L, et al. Application of hybrid genetic algorithm in optimal Lambert orbital transfer design[J]. *Flight Dynamics*, 2013, 31(3): 269–272.
- [13] JACKSON B. Minimum energy maneuvering strategies for multiple satellite inspection missions[C]// Proc. of the 2005 IEEE Aerospace Conference, 2005: 4306–4313.
- [14] PENG C Y, ZHANG J, YAN B, et al. Multisatellite flyby inspection trajectory optimization based on constraint repairing[J]. *Aerospace*, 2021, 8(9): 274.
- [15] GATTO G, CASALINO L. Fast evaluation and optimization of low-thrust transfers to multiple targets[J]. *Journal of Guidance, Control, and Dynamics*, 2015, 38(8): 1525–1530.
- [16] ZHU Y H, LUO Y Z. Fast evaluation of low-thrust transfers via multi-

- layer perceptions[J]. *Journal of Guidance, Control, and Dynamics*, 2019, 42(12): 2627–2637.
- [16] BANDO M, YAMAKAWA H. Orbital design for multiple flyby missions[J]. *Transactions of the Japan Society for Aeronautical and Space Sciences, Aerospace Technology Japan*, 2010, 8(27): 9–13.
- [17] KONG L J, ZHOU Y. Optimal planning of multiple geosynchronous spacecraft flyby inspection mission with multiple servicing space robots[J]. *International Journal of Aeronautical and Space Sciences*, 2024, 25: 1178–1193.
- [18] ZHOU Y, YAN Y, HUANG X, et al. Mission planning optimization for multiple geosynchronous satellites refueling[J]. *Advances in Space Research*, 2015, 56(11): 2612–2625.
- [19] RAKHIMOV M, ZARIPOVA D, JAVLIEV S. Deep learning parallel approach using CUDA technology[C]// *Proc. of the Intensive Scientific Conference on Modern Problems of Applied Science and Engineering*, 2024.
- [20] ABDULKADIR G, OKAN O, CENK G. CNN-LSTM based deep learning application on Jetson Nano: estimating electrical energy consumption for future smart homes[J]. *Internet of Things*, 2024, 26: 101–148.
- [21] DIPTARUP S, KARAN D, NARENDRA P, et al. Implementation of image enhancement algorithms and recursive ray tracing using CUDA[J]. *Procedia Computer Science.*, 2016, 79: 516–524.
- [22] ZHAO S W, LAI Z S, ZHAO J D. Leveraging ray tracing cores for particle-based simulations on GPUs[J]. *International Journal for Numerical Methods in Engineering*, 2023, 124(3): 696–713.
- [23] TAN H, XIAO H, LIU Y, et al. LASSO approximation and application to image super-resolution with CUDA acceleration [C]// *Proc. of the 2nd International Conference on Image, Vision and Computing*, 2017: 483–488.
- [24] DAWA C L, BHAWNA G, AYUSH D, et al. Image super-resolution: a comprehensive review, recent trends, challenges and applications[J]. *Information Fusion*, 2023, 91: 230–260.
- [25] LIU H, HE J R, MAO H X, et al. A CUDA based infrared image simulation method of rocket plumes observed by satellite [C]// *Proc. of the 7th Asia Pacific Conference on Optics Manufacture and 2021 International Forum of Young Scientists on Advanced Optical Manufacturing*, 2022: 487–492.
- [26] ALESSANDRO M, CAMILLA C, ARNAUD B. GPU-based high-precision orbital propagation of large sets of initial conditions through Picard–Chebyshev augmentation[J]. *Acta Astronautica*, 2023, 204: 239–252.
- [27] NITIN A, RYAN P R, VIVEK V. Parallel computation of multiple space trajectories using GPUs and interpolated gravity models[J]. *Journal of Guidance Control, and Dynamic*, 2015, 38(8): 1457–1470.
- [28] UEDA S, OGAWA H. Multi-fidelity approach for global trajectory optimization using GPU-based highly parallel architecture[J]. *Aerospace Science and Technology*, 2021, 116: 106829.
- [29] GIACOMO A, ATILIM G B, DARIO I. Closing the gap between SGP4 and high-precision propagation via differentiable programming[J]. *Acta Astronautica*, 2025, 226(1): 694–701.
- [30] KIM Y, SHRIVASTAVA A. Memory performance estimation of CUDA programs[J]. *ACM Transactions on Embedded Computing Systems*, 2013, 13(2): 21.
- [31] LUCAS P, MARK P, WOUTER E, et al. A fast three-dimensional gamma evaluation using a GPU utilizing texture memory for on-the-fly interpolations[J]. *Medical Physics.*, 2011, 38(7): 4032–4035.
- [32] JOSHUA H, USMAN A. FastSpline: automatic generation of interpolants for lattice samplings[J]. *ACM Transactions on Mathematical Software*, 2023, 49(2): 12.

作者简介

刘浩(1993—),男,工程师,博士,主要研究方向为航天器系统工程以及任务规划。

贺子鹏(1996—),男,工程师,硕士,主要研究方向为航天器系统工程以及任务规划。

徐广德(1990—),男,高级工程师,博士,主要研究方向为航天器总体设计。